On The Porch

Current vs. Proposed Architecture

A Beginner-Friendly, Systematic Comparison

Technical Design Document

June 2026

Abstract

This document compares two ways to build **On The Porch**, a Dorchester community chatbot: the **current implementation** in this repository and a **proposed greenfield design** optimized for 130,000 residents, sub-5-second responses, and production reliability. It is written for readers who are new to the project. We explain key concepts first, then describe each architecture separately, then compare feature-by-feature with clear tables and plain language. Finally, we explain *why* the current system underperforms in specific areas and how it can be improved.

Contents

I Foundations for Beginners 4

1 What Is This App? 4

2 Glossary of Terms 4

3 How to Read This Document 5

II Current Project Architecture 5

4 High-Level Picture (Current) 5
4.1 Request Flow (Current Chat) . . . 5

5 Current: Frontend 6

6 Current: Authentication 6

7 Current: Data Storage 6

8 Current: Ingestion Pipeline 7

9 Current: Chat Intelligence Details 7
9.1 Routing (`_route_question`) . . . 7
9.2 Follow-up gate (`_check_if_needs_new_data`) . . . 7
9.3 SQL path (`sql_chat/app4.py`) . . . 7
9.4 RAG path (`retrieval.py`) . . . 7
9.5 Caching . . . 8

10 Current: Events Panel **8**

11 Current: Admin Dashboard **8**

12 Current: Deployment & Operations **8**

III Proposed Greenfield Architecture **8**

13 High-Level Picture (Proposed) **8**
13.1 Request Flow (Proposed Chat) 9

14 Proposed: Data Layer **9**

15 Proposed: Ingestion **9**

16 Proposed: Auth, Events, Admin **9**

IV Side-by-Side Comparison **10**

17 Architecture Overview **10**

18 Chat Pipeline — The Most Important Difference **10**

19 Feature-by-Feature Comparison **11**
19.1 Chat / Conversations 11
19.2 Authentication 11
19.3 Events Panel 11
19.4 Admin Dashboard 12
19.5 Data Ingestion 12
19.6 Retrieval & Answer Quality 12

V Performance, Scalability, and Complexity **13**

20 Latency — Why the Current App Feels Slow **13**
20.1 Root causes of current latency 13

21 Scalability — 130,000 Residents **13**

22 Complexity — Ease of Building and Maintaining **14**

23 Reliability and Data Freshness **14**

24 Security **14**

25 Cost (Gemini API) **15**

VI Why the Current Project Underperforms **15**

26 Summary Table: Gaps **15**

27 What the Current Project Already Does Well **15**

VII Improvement Roadmap (Without Full Rewrite) 15

28 Phased Migration: Current → Proposed 16
28.1 Phase A — Quick latency wins (1–2 weeks) 16
28.2 Phase B — Routing simplification (2–4 weeks) 16
28.3 Phase C — Vector DB migration (4–8 weeks) 16
28.4 Phase D — Data plane hardening (ongoing) 16

29 Decision Guide 17

VIII Conclusion 17

30 One-Paragraph Summary 17

31 Further Reading in This Repository 17

Part I
Foundations for Beginners

1 What Is This App?

On The Porch is a website where Dorchester residents ask questions in plain English and receive answers grounded in local data: neighborhood news, 311 service requests, 911/crime statistics, community events, policy documents, and more.

Think of it as three things working together:

1. A **website** (buttons, chat box, events sidebar) that users see in the browser.
2. A **backend API** (a program running on a server) that receives questions, looks up data, and calls an AI model.
3. A **data layer** (databases and search indexes) that stores civic data and documents, kept fresh by scheduled **ingestion jobs**.

2 Glossary of Terms

Frontend The code that runs in the user's browser: HTML, CSS, JavaScript. Files live in `public/`.

Backend / API The server program that handles login, chat, events, and admin. Here: `api/api_v2.py` (Flask).

LLM Large Language Model — the AI (e.g. Gemini) that reads context and writes natural-language answers.

MySQL A relational database for structured data: users, chat threads, 311 rows, events tables.

Vector database A search index for *meaning* — finds document chunks similar to a question even if words differ. Current project uses **ChromaDB** (SQLite file on disk).

RAG Retrieval-Augmented Generation — fetch relevant documents first, then ask the LLM to answer using only those documents.

SQL path Answer by running a database query (counts, lists, trends) instead of searching documents.

Hybrid Run both SQL and RAG, then merge results with another LLM call.

Ingestion Background jobs that download/sync data from Boston Open Data, RSS, Google Drive, etc., into MySQL and the vector store.

SSE / Streaming Server-Sent Events — the server sends answer tokens one-by-one so the user sees text appear progressively.

Redis An in-memory cache server shared across multiple API workers (sessions, rate limits, optional answer cache).

Gunicorn A production web server that runs multiple copies (workers) of the Flask app for concurrency.

Guest session A temporary user identity stored in the browser tab — no password required.

3 How to Read This Document

- Orange-tinted boxes
  summarize the **current** project.
- Blue-tinted boxes
  summarize the **proposed** design.
- Comparison tables use **Current** and **Proposed** columns.
- "LLM calls" means round-trips to Gemini — each adds roughly 0.5–3 seconds of latency and costs money.

Part II
Current Project Architecture

4 High-Level Picture (Current)

Stack: Vanilla JS frontend → Flask API (`api_v2.py`) → `unified_chatbot.py` → MySQL + ChromaDB + Gemini API.
Deployment: Can run as Flask dev server or Gunicorn (`start_api.sh`). Optional local Redis (`start_redis.sh`). Static frontend via `python -m http.server`.
Repo layout: `public/` (UI), `api/` (Flask), `on_the_porch/unified_chatbot.py` (brain), `sql_chat/app4.py` (SQL), `rag stuff/retrieval.py` (vectors), `data_ingestion/` (cron).

4.1 Request Flow (Current Chat)

When a user sends a message, this is the typical path:

1. Browser calls `POST /conversations/{id}/messages/stream` (SSE) or non-streaming POST.
2. Flask authenticates (cookie session or guest Bearer token), checks CSRF, rate limits.
3. API loads thread history from MySQL, releases DB connection.
4. `unified_chatbot` runs:
   (a) Small-talk check (instant, no LLM) — or continue.
   (b) Optional semantic cache hit (embedding similarity) — or continue.
   (c) If follow-up: **LLM call #1** — "does this need new data?" (`_check_if_needs_new_data`)
   (d) If can use cache/history: **LLM call** — answer from history — done.
   (e) Else: **LLM call #2** — route to sql/rag/hybrid (`_route_question`)
   (f) Execute mode:
   - **SQL:** pick tables (LLM) → generate SQL (LLM) → run MySQL → summarize (LLM). Events fast-path skips some steps for generic calendar questions.
   - **RAG:** parallel retrieval from 4 sources → compose answer (LLM, streamed).
   - **Hybrid:** SQL + RAG in parallel → merge (LLM).

   (g) Non-streaming: Boston.gov fallback may add scrape + classifier + regeneration.
   (h) Streaming: fallback runs *after* stream via `correction` event (recent improvement).
5. API saves messages to MySQL, logs interaction, returns response.

Typical LLM count: 3–6 calls per non-trivial question. **Typical latency:** 5–30+ seconds for complex hybrid questions; 1–3s for chitchat or cache hits.

```
Browser -> api_v2.py -> unified_chatbot.py
              |              |-- is_small_talk? (0 LLM)
              |              |-- semantic cache? (0-1 embed)
              |              |-- needs_new_data? (1 LLM)
              |              |-- route_question (1 LLM)
              |              |-- sql / rag / hybrid (1-3 LLM)
              |              |-- boston.gov fallback (0-2 LLM, non-stream)
              v
        MySQL + ChromaDB + Gemini
```

5 Current: Frontend

- **Technology:** Plain HTML/CSS/JS — no React framework.
- **Files:** `public/app.js` (UI logic), `public/api.js` (HTTP client), `public/config.js`.
- **Guest-by-default:** On load, `ensureSession()` creates guest via `POST /auth/guest`; stores Bearer token in `sessionStorage`.
- **Registered users:** HttpOnly cookies + CSRF header on mutations.
- **Streaming:** Enabled by default; SSE with 120s client timeout (recent addition).
- **Instant replies:** `instantReplies.js` shows chitchat locally before server confirms.
- **Events panel:** Fetches `GET /events` on load — correctly decoupled from chat LLM.
- **Admin:** Separate `admin.html` page; checks session and blocks guests.

6 Current: Authentication

- Password signup/login (Argon2 hashing).
- Google OAuth **removed** in recent work.
- Guest users: ephemeral DB row + Bearer session (24h TTL), tab-scoped.
- Admin promotion via `AUTH_ADMIN_EMAILS` env list (never for guests).
- Rate limits: guest creation per IP; chat per session (Redis if configured, else per-worker memory).
- Guest cleanup cron script (`cleanup_guest_users.py`) — opt-in via env flag.

7 Current: Data Storage

Store	Used for
MySQL	Users, sessions, threads, messages, 311/911 tables, events, admin knowledge, interaction l
ChromaDB (`vectordb_new/`)	Document embeddings (news, policies, transcripts) — SQLite-backed files
Redis (optional)	Session cache for legacy `/chat`, rate limits, semantic cache is *in-process only*
Flask session / memory	Per-worker caches when Redis not enabled

Table 1: Current data stores

Pain point: Chroma uses a local SQLite file. When the ingestion cron writes while users read, SQLite can lock — retrieval retries with WAL mode, but contention remains at scale.

8 Current: Ingestion Pipeline

`cron_ingest.sh` runs sequentially (with `flock` lock):

1. Boston 311 & 911 sync
2. DotNews newsletter ingest
3. Tribe events
4. RSS via `ingest_rss_wayback.py` (can be slow — Wayback backfill)
5. Google Drive → vector DB
6. Guest user cleanup

MySQL updates are incremental (upsert) — safe for live readers. Vector writes go *directly* into the live Chroma store (no blue-green swap).

9 Current: Chat Intelligence Details

9.1 Routing (`_route_question`)

- Large prompt with many mandatory rules sent to `GEMINI_FAST_MODEL`.
- Returns JSON: `{mode, transcript_tags, policy_sources, k}`.
- JSON parse failure → defaults to **hybrid** (slowest path).

9.2 Follow-up gate (`_check_if_needs_new_data`)

- Separate LLM call on every follow-up when history/cache exists.
- Prompt bias: "when in doubt, fetch new data" — increases latency but reduces stale answers.

9.3 SQL path (`sql_chat/app4.py`)

- Schema snapshot (cached with TTL — improvement added).
- LLM picks relevant tables.
- LLM generates SQL string.
- Executes on MySQL with retries.
- LLM summarizes results into natural language.
- **Fast path:** `run_generic_events_query()` for "events this week" — direct SQL, fewer LLM calls.

9.4 RAG path (`retrieval.py`)

- Four parallel retrievals: RSS, Boston.gov cache, transcripts, policies.
- Each embeds the query separately (Gemini embedding API).
- Chroma similarity search; on lock failure, keyword scan over *entire* collection (expensive).
- LLM composes final answer from top chunks.

9.5 Caching

- **Semantic cache** (`api/semantic_cache.py`): off by default; in-process per worker; first-turn only.
- **Thread state JSON:** stores last SQL/RAG results for follow-ups.
- **Legacy /chat session cache:** Redis or memory — not used by main conversation API path.

10 Current: Events Panel

- `GET /events?days_ahead=14&limit=10` reads MySQL `weekly_events` (or similar).
- No LLM involved — good design.
- Panel loads in parallel with auth — good design.
- Client-side event card layout with ResizeObserver (recent bug fix for `eventsPanel`).

11 Current: Admin Dashboard

- `public/admin.html` — stats, flagged interactions, knowledge CRUD, community notes chat.
- Protected by `_require_admin()` + guest block.
- CSRF on mutating routes (recent security hardening).
- Knowledge changes trigger background Chroma re-ingest thread.

12 Current: Deployment & Operations

- **Dev:** `python api/api_v2.py` (Flask threaded).
- **Prod-capable:** `./start_api.sh` → Gunicorn, 3 workers × 8 threads.
- **Redis:** Optional; required for correct rate limits across workers.
- **Health:** `/health` checks MySQL, Redis, Chroma (recent expansion).
- **Observability:** Print logging; no structured metrics/tracing by default.

Part III
Proposed Greenfield Architecture

13 High-Level Picture (Proposed)

Stack: Modern SPA (React or Vite JS) → FastAPI/Flask behind load balancer → single **Chat Orchestrator** → PostgreSQL + Qdrant + Redis + Gemini.
Key difference: Cap LLM calls at 1–2 per question. Precompute aggregates. Decouple all non-chat surfaces from the LLM.
Deployment: Containerized API pods (autoscaling), managed PostgreSQL, managed Redis cluster, Qdrant Cloud or self-hosted vector DB, CDN for static assets, separate ingestion worker fleet.

13.1 Request Flow (Proposed Chat)

1. **Tier 0** (<100ms): Rules, exact cache, FAQ, chitchat — **zero LLM**.
2. **Tier 1** (<200ms): Hybrid intent classifier (keywords + regex + tiny model) — **not** a large generative routing prompt.
3. **Tier 2** (<800ms): Parallel data fetch:
   - Events/311/911 common questions → **prepared SQL** (no LLM SQL generation).
   - News/policy → parallel BM25 + vector search.
4. **Tier 3** (<2.5s): **One** synthesis LLM call with structured context + citations, streamed.
5. **Async enrichment:** Boston.gov / external sources only if needed, *after* first answer — never blocking p95.

Typical LLM count: 0–2. **Target latency:** p50 1.5–2.5s, p95 $\leq$ 5s.

14 Proposed: Data Layer

- **PostgreSQL** for OLTP + materialized views (`events_materialized`, `mv_311_weekly_by_neighborhood`)
- **Qdrant or pgvector** instead of file-based Chroma — no SQLite lock contention.
- **Redis cluster** mandatory for sessions, rate limits, L1/L2/L3 caches — shared across all API replicas.
- **S3** for raw PDFs and ingestion artifacts.
- **Blue-green vector indexing** — bulk rebuilds swap alias, never lock live readers.

15 Proposed: Ingestion

- Job queue (Redis/SQS) instead of single bash script only.
- 311/911 every 30–60 min; news every 1–4 hours; events daily.
- Wayback backfill as **offline batch**, not in daily cron.
- Ingestion lag exposed in `/health` with alert thresholds.
- Materialized view refresh after civic data sync — events API reads views only.

16 Proposed: Auth, Events, Admin

Same *features* as current, stricter *separation*:

- Guest + password auth (same product semantics).
- Events API: 100% SQL on materialized table, p95 <100ms, ETag caching.
- Admin: read replica for heavy stats queries; immutable audit log; same RBAC model.

Part IV
Side-by-Side Comparison

17 Architecture Overview

Aspect	Current	Proposed
Overall pattern	Monolithic Flask app importing a large Python chatbot module	Stateless API pods + dedicated orchestrator + async workers
Frontend	Vanilla JS SPA, static file server	CDN-hosted SPA (React/Vite)
API framework	Flask + Gunicorn	FastAPI or Flask behind LB + autoscaling
Primary database	MySQL	PostgreSQL (+ optional Timescale for 311/911)
Vector search	ChromaDB (local SQLite files)	Qdrant or pgvector (managed, concurrent)
Cache	Optional Redis; many caches in-process	Redis cluster required
LLM provider	Google Gemini (multiple models)	Same, but fewer calls
Ingestion	Bash cron script, sequential steps	Queue + worker fleet + blue-green index
Auth	Guest Bearer + password cookies	Same semantics, hardened at scale

Table 2: Architecture overview comparison

18 Chat Pipeline — The Most Important Difference

Step	Current	Proposed
1. Chitchat	Rule-based, 0 LLM (good)	Rule-based + FAQ table, 0 LLM
2. Cache check	Semantic cache optional, per-worker, off by default	L1 exact + L2 semantic + L3 intent cache in Redis
3. Follow-up handling	Extra LLM: "needs new data?"	Thread state + rules; no extra LLM gate
4. Routing	Large LLM prompt → sql/rag/hybrid JSON	Keyword/rules + light classifier; fixed 12 intents
5. SQL questions	LLM picks tables + LLM writes SQL + LLM summarizes	Prepared query catalog for common intents; LLM SQL only for novel queries with 2s timeout
6. RAG retrieval	4 parallel Chroma searches; embed per search	BM25 + vector fusion + optional rerank; embeddings batched at ingest
7. Hybrid	SQL + RAG parallel, then LLM merge	Same parallelism, but single synthesis call with structured JSON
8. Boston.gov fallback	Blocks non-streaming path; async correction in streaming (recent)	Always async after first answer

Step	Current	Proposed
9. Streaming	SSE; SQL pseudo-streams after full compute	SSE from first token; same
10. LLM calls (typical)	**3–6**	**0–2**
11. Latency (typical)	5–30s complex; 1–3s simple	Target 1.5–5s

Table 3: Chat pipeline comparison

19 Feature-by-Feature Comparison

19.1 Chat / Conversations

Aspect	Current	Proposed
Thread storage	MySQL `conversation_threads` + messages	PostgreSQL equivalent
Thread state	JSON blob with last SQL/RAG cache	Structured state with intent + doc IDs
Streaming	Yes, default on	Yes, required
Sources/citations	Extracted post-answer from mode result	Mandatory in synthesis prompt
Message limits	8000 chars (recent)	2000 chars (stricter default)
Concurrent sends	Client lock (`isSendingMessage`)	Server-side thread lock recommended
Flag flow	PUT /log with ownership check	Same pattern

19.2 Authentication

Aspect	Current	Proposed
Guest model	Bearer + sessionStorage, tab-scoped	Same
Registered	HttpOnly cookie + CSRF	Same
OAuth	Removed	Optional later, not required
Admin gate	API + admin.html bootstrap	Same + audit log
Rate limits	Redis or per-worker memory	Redis cluster only
Guest cleanup	Cron script, opt-in	Cron + metrics on guest table size

19.3 Events Panel

Aspect	Current	Proposed
Data source	MySQL events tables directly	Materialized `events_materialized` view
LLM involved?	No (good)	No

Aspect	Current	Proposed
Prefetch	On app load	On app load + ETag
Latency	Fast (<200ms typical)	Target <100ms p95
Chat coupling	None	None

Verdict: Events panel is already well-designed in the current project. Proposed design mainly adds materialized views and HTTP caching.

19.4 Admin Dashboard

Aspect	Current	Proposed
UI	Single `admin.html`	Could remain or split into admin SPA
Stats queries	Live aggregation on `interaction_log`	Pre-aggregated counters + read replica
Knowledge CRUD	MySQL + background Chroma ingest	MySQL + queue-based embed worker
Community notes	Approve flow exists	Same + explicit job queue
Security	CSRF + admin role (recent hardening)	Same + immutable audit log

19.5 Data Ingestion

Aspect	Current	Proposed
Orchestration	`cron_ingest.sh` bash	Scheduler + job queue
Overlap protection	flock file lock	Queue deduplication + worker leases
311/911	Incremental upsert (good)	Same + materialized view refresh
News/RSS	Includes Wayback script in daily cron	RSS frequent; Wayback offline only
Vector writes	Direct to live Chroma	Incremental upsert + blue-green rebuild
Freshness visibility	Partial (/health expanded)	Explicit lag metrics per source
Failure handling	Log file	Alerts + dead-letter queue

19.6 Retrieval & Answer Quality

Aspect	Current	Proposed
Search method	Vector similarity (Chroma) + keyword fallback	Hybrid BM25 + vector + RRF fusion
Embeddings	Per-query via Gemini API	Primarily at ingest; query embed once
Chunking	Per ingest script	Standardized sizes per doc type
Grounding	Prompt instructions + sources list	Strict context contract + citation IDs

Aspect	Current	Proposed
Safety (911)	Answers generated	Aggregation rules + suppression thresholds
Hallucination risk	Higher when SQL LLM wrong	Lower with prepared queries + citations

Part V
Performance, Scalability, and Complexity

20 Latency — Why the Current App Feels Slow

Question type	Current (approx.)	Proposed (target)
"Hi" / chitchat	0.1–2s (instant UI + persist)	<0.1s
"Events this week"	2–8s (fast SQL path helps)	0.2–1s (zero LLM)
"311 requests last month"	8–20s (full SQL pipeline)	1–3s (prepared SQL)
News / policy question	10–25s (RAG + routing)	2–5s
Hybrid neighborhood question	15–40s (slowest path)	3–5s
Cache hit (rare today)	1–3s	<0.5s

Table 10: Latency comparison by question type

20.1 Root causes of current latency

1. **Too many LLM round-trips** — routing + needs-new-data + SQL gen + summarize + merge.
2. **Hybrid as default fallback** — when routing JSON fails, hybrid runs both SQL and RAG plus merge.
3. **LLM-generated SQL** — even when a fixed query template would work.
4. **Sequential embedding** — multiple embed API calls per RAG question.
5. **Boston.gov on hot path** — scraping + classifier + regeneration (non-streaming).
6. **Chroma lock retries** — ingestion contention adds seconds occasionally.

21 Scalability — 130,000 Residents

Factor	Current	Proposed
API horizontal scale	Gunicorn multi-worker works, but caches diverge without Redis	Stateless pods + shared Redis — linear scale
Rate limiting	Broken across workers without Redis (3× effective limit)	Correct global limits
Vector DB concurrency	Chroma SQLite — poor write/read concurrency	Qdrant/pgvector — built for concurrent reads
DB connections	MySQL pool per worker (fixed)	PgBouncer + pool tuning

Factor	Current	Proposed
Peak 2000 concurrent	Risky without Redis + Qdrant migration	Designed for it
Ingestion vs live traffic	Same Chroma file — lock risk	Separate index build path
Cost at scale	High Gemini bill (many calls/user)	Lower bill (fewer calls + cache hits)

Table 11: Scalability comparison

22 Complexity — Ease of Building and Maintaining

Factor	Current	Proposed
Code organization	Large `unified_chatbot.py` (∼1500 lines) + Flask API (∼3200 lines)	Smaller orchestrator + query catalog + workers
Folder naming	`rag stuff/` (space in path)	Clean `rag/` module
Routing logic	200+ line LLM prompt	Intent table + rules — easier to test
SQL logic	Dynamic LLM SQL — hard to predict	Named queries — auditable
Deploy steps	Manual: Redis, API, frontend, cron	Containers + CI/CD + health gates
Debugging	Print statements	Structured traces per request
Onboarding new dev	Must understand full LLM chain	Can work on ingest OR API OR frontend separately

Table 12: Complexity comparison

23 Reliability and Data Freshness

Factor	Current	Proposed
311/911 freshness	Cron-dependent; no SLA metric	30–60 min target + alert if stale
News freshness	RSS in cron; Wayback may delay	1–4 hour target; Wayback offline
Vector index safety	Live writes during cron	Blue-green swap
MySQL gone-away	Fixed with pool ping (improvement)	Same pattern on PostgreSQL
Error responses	JSON error handlers added (improvement)	Built-in from day one
Degraded mode	Partial — keyword fallback	Explicit stale-cache + disclaimer

24 Security

Both designs converge after recent hardening (CSRF on admin, log ownership, guest blocks). Proposed adds: secret manager, audit log, stricter input limits by default, synthetic security monitor-

ing.

25 Cost (Gemini API)

Rough model: Each LLM call ≈ $0.001–0.01 depending on tokens.

- **Current:** 50,000 questions/month × 4 calls avg = 200,000 API calls.
- **Proposed:** 50,000 questions/month × 1.2 calls avg (with 25% cache) = 60,000 API calls.

Proposed design reduces cost roughly 60–70% at the same traffic, while improving latency.

Part VI
Why the Current Project Underperforms

26 Summary Table: Gaps

Area	Symptom users feel	Root cause in current design
Chat speed	Long "Thinking…"	3–6 LLM calls per question
Inconsistent speed	Sometimes fast, sometimes 30s+	Hybrid fallback + Chroma locks
Answer quality gaps	"Couldn't find info" on streaming	Boston.gov was skipped in stream (partially fixed)
Production deploy	Rate limits ineffective	In-memory cache per Gunicorn worker
Ingestion risk	Occasional search failures during cron	Chroma SQLite concurrent read/write
Ops blind spots	Hard to know if data is stale	No per-source freshness SLAs
Developer velocity	Hard to change routing rules	Logic buried in giant LLM prompts

27 What the Current Project Already Does Well

Do **not** throw everything away. These parts are solid or near-optimal:

1. **Events panel decoupled from LLM** — keep as-is; add materialized views.
2. **Guest-by-default auth model** — product semantics are correct.
3. **Small-talk bypass** — matches proposed Tier 0.
4. **Parallel RAG retrieval** — matches proposed parallelism.
5. **Events fast SQL path** — prototype of proposed prepared queries.
6. **MySQL incremental ingest** — safe; keep pattern.
7. **Streaming SSE UX** — correct approach.
8. **Admin feature set** — meets requirements; needs scale hardening only.
9. **Recent reliability fixes** — pool ping, WAL, flock, gunicorn, security patches.

Part VII
Improvement Roadmap (Without Full Rewrite)

28 Phased Migration: Current → Proposed

You can move toward the proposed architecture incrementally:

28.1 Phase A — Quick latency wins (1–2 weeks)

1. Add **prepared SQL catalog** for top 10 intents (311 weekly, 911 monthly, events list).
2. Skip `_check_if_needs_new_data` LLM when thread state has high-confidence match.
3. Enable **semantic cache** in Redis (not in-process) with `SEMANTIC_CACHE_ENABLED=true`.
4. Remove `ingest_rss_wayback.py` from daily cron; run offline monthly.
5. Require **Redis** in production (`CACHE_BACKEND=redis`).

Expected impact: 30–50% latency reduction on common questions.

28.2 Phase B — Routing simplification (2–4 weeks)

1. Replace LLM routing with rules + lightweight classifier for 12 intents.
2. Change hybrid JSON-parse failure default to `rag` or `sql`, not hybrid.
3. Move Boston.gov fallback fully off hot path (already started for streaming).

Expected impact: More predictable latency; fewer 30s+ outliers.

28.3 Phase C — Vector DB migration (4–8 weeks)

1. Migrate Chroma → Qdrant or pgvector.
2. Implement blue-green index builds for bulk ingest.
3. Add BM25 keyword search alongside vectors.

Expected impact: Eliminate lock contention; better recall for street names.

28.4 Phase D — Data plane hardening (ongoing)

1. Materialized views for events and 311 aggregates.
2. Ingestion job queue with lag metrics and alerts.
3. Read replica for admin stats.
4. Structured request tracing (intent, tier, timings).

If your priority is...	Do this first
Faster answers this month	Phase A (prepared SQL + Redis cache)
Fewer production incidents	Redis required + Chroma migration (Phase C)
Lower Gemini bill	Reduce LLM calls (Phase A + B)
130k-resident scale	Full proposed data plane (Phases C + D)
Ship features faster	Phase B (simpler routing code)

29 Decision Guide

Part VIII
Conclusion

30 One-Paragraph Summary

The **current On The Porch** is a capable monolith: it combines Flask, MySQL, ChromaDB, and multiple Gemini calls to route questions through SQL, RAG, or hybrid paths. It works and has many good patterns (guest auth, events decoupling, streaming, parallel retrieval), but it is **latency-limited by design** because it asks the AI too many times per question and stores vectors in a file database that fights with ingestion jobs.

The **proposed architecture** keeps the same user-facing features but reorganizes the *hot path*: classify intent cheaply, fetch data in parallel with prepared queries, synthesize once, cache aggressively, and move slow enrichment off the critical path. It trades prompt complexity for system structure — making the app faster, cheaper, and easier to scale to Dorchester's full population.

31 Further Reading in This Repository

- `docs/system_design.pdf` — full greenfield design specification.
- `changes.md` — chronological log of improvements already made to the current codebase.
- `README.md` — how to run the current project locally.